

Exactness Properties of LCN Marginal-Inference Algorithms

A consolidated reference: what each engine bounds, when it is exact, outer, inner, or unreliable — with formal results and proofs

IBM Research

Abstract

The LCN library ships nine marginal-inference engines — the exact oracle **ExactInference**, the junction-tree NLP **CredalJT** (scheme D5), the credal-network family Credal VE, Credal CTE, ApproxLP, Interval BP, Credal IJGP, and the structural engines ARIEL and SCC-factor-graph propagation. Their exactness behavior has been analyzed one engine at a time in six companion notes (**ariel_exactness**, **cve_exactness**, **ccte_exactness**, **ibp_exactness**, **approxlp_exactness**, **tighter_approximation**). This document consolidates that analysis into a single reference. We fix one framework — the LCN distribution set $\mathcal{D}_{\text{LCN}}$, its strong extension $\mathcal{D}_{\text{SE}} \supseteq \mathcal{D}_{\text{LCN}}$, and the two looseness sources (Gap A, Gap B) — and then place every engine in it: which feasible set it optimizes, whether its returned interval is *exact*, a valid *outer* bound, an *inner* bound (hence *not* a valid outer bound), or *unreliable* (a solver artifact); the precise topological condition under which it is exact; and its cost. We state the formal results with proof sketches, collected in one place: the fundamental inclusion $\mathcal{D}_{\text{LCN}} \subseteq \mathcal{D}_{\text{SE}}$; CVE’s exactness over $\mathcal{D}_{\text{SE}}$; the exactness of CVE/IBP/ARIEL on clean singly-connected LCNs; ApproxLP’s inner-bound guarantee; IBP’s mode dichotomy; IJGP’s exactness at i -bound $\geq$ treewidth; ARIEL’s structural LMC preservation; and the D5/ CredalJT exactness theorem (hypotheses H1–H4, with the projection/reconstruction proof). Throughout we separate three distinct kinds of looseness — *definitional* (an engine that honestly targets $\mathcal{D}_{\text{SE}}$), *solver* (ipopt local optima in **ExactInference**), and *implementation* (the now-fixed ARIEL factor-grouping bug; the two now-fixed Interval BP message bugs) — and report the *shipped* reality, not merely the intended theory. A master table and a topology cross-cut table give the one-screen summary, and a closing decision guide says which engine to reach for.

Contents

1	The unifying framework	2
1.1	LCNs, the LCN distribution set, and the marginal task	2
1.2	The credal network and the strong extension	3
1.3	The two looseness sources	3
1.4	The bound-character taxonomy	4
1.5	Three kinds of looseness	4
2	Master table	4
3	The exact family: targeting $\mathcal{D}_{\text{LCN}}$	4
3.1	ExactInference : the definitional oracle	4
3.2	CredalJT (D5): exact at treewidth cost	5

4	The credal-network family: targeting $\mathcal{D}_{SE}$	7
4.1	Credal VE: exact over the strong extension	7
4.2	Credal CTE: exact over $\mathcal{D}_{SE}$, but order-sensitive	7
4.3	ApproxLP: an inner bound	8
4.4	Interval BP: outer / inner by mode; exact on trees	8
4.5	Credal IJGP: exact at i -bound $\geq$ treewidth	9
5	Structural and hybrid engines: targeting $\mathcal{D}_{LCN}$ by decomposition	9
5.1	ARIEL: structural LMC preservation	9
5.2	SCCP / SCCP-ARIEL: exactness on the acyclic condensation	9
6	Topology cross-cut	10
7	Reliability and verification	10
8	Decision guide	11

1 The unifying framework

1.1 LCNs, the LCN distribution set, and the marginal task

An LCN over binary atoms $\mathbf{V} = \{V_1, \dots, V_n\}$ is a set of probability *sentences*, each Type-1 $\ell \leq P(\varphi) \leq u$ or Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$ over propositional formulas, together with the conditional independencies of the *Local Markov Condition* (LMC) of its primal graph. A candidate joint distribution $p \in \Delta(2^{\mathbf{V}})$ over the 2^n truth assignments belongs to the *LCN distribution set* $\mathcal{D}_{LCN}$ when it meets two families of requirements:

- *Sentence bounds.* Each sentence s constrains one probability of p to its interval, $\ell_s \leq g_s(p) \leq u_s$, where $g_s(p) = P(\varphi)$ for a Type-1 sentence and $g_s(p) = P(\varphi \mid \psi)$ (a ratio of two marginals of p) for a Type-2 sentence.
- *LMC equalities.* Each LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ imposes, for *every* joint configuration $\mathbf{x}, \mathbf{y}, \mathbf{s}$ of the atom blocks X, Y, S , the bilinear equality

$$P(\mathbf{x}, \mathbf{y}, \mathbf{s}) P(\mathbf{s}) = P(\mathbf{x}, \mathbf{s}) P(\mathbf{y}, \mathbf{s}), \quad (1)$$

i.e. $X \perp\!\!\!\perp Y \mid S$ under p . We write $h_\iota(p) = 0$ for the collection of these equalities belonging to assertion ι .

Thus

$$\mathcal{D}_{LCN} = \left\{ p \in \Delta(2^{\mathbf{V}}) : \ell_s \leq g_s(p) \leq u_s \text{ for all sentences } s, \quad h_\iota(p) = 0 \text{ for all LMC assertions } \iota \right\}. \quad (2)$$

The marginal task for a query atom a is to compute its tightest lower and upper probability,

$$\underline{P}(a) = \min_{p \in \mathcal{D}_{LCN}} P(a), \quad \overline{P}(a) = \max_{p \in \mathcal{D}_{LCN}} P(a),$$

and, when evidence e is given, the analogous bounds on the conditional $P(a \mid e)$.

Notation (restriction). For an atom set $A \subseteq \mathbf{V}$, write $p \upharpoonright A$ for the *marginal of p on A* — the distribution over the truth assignments of A obtained by summing p over the remaining atoms. Every constraint above is *local*: it depends on p only through one such marginal — sentence s through $p \upharpoonright \text{atoms}(s)$, and assertion $(X \perp\!\!\!\perp Y \mid S)$ through $p \upharpoonright (X \cup Y \cup S)$. Locality is the lever behind every exactness result below.

1.2 The credal network and the strong extension

When the LCN’s structure graph is a chain graph, the joint factorizes over families:

$$P(\mathbf{V}) = \prod_v P(\text{ch}_v \mid \text{pa}_v), \quad (3)$$

and this factorization reproduces exactly the LMC independencies. The `cn/` pipeline (`CredalNetworkVertices.from_lcn`) compiles the LCN into a *credal network*: interval-valued local credal sets K_v with enumerated extreme points. The credal-network engines bound not $\mathcal{D}_{\text{LCN}}$ but the *strong extension*

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_v \text{ chosen independently per parent-config} \right\}. \quad (4)$$

Proposition 1 (Fundamental inclusion). $\mathcal{D}_{\text{LCN}} \subseteq \mathcal{D}_{\text{SE}}$. *The inclusion is equality when the primal graph is singly connected and there is no cross-family sentence (a sentence whose atom scope lies in no single family); it is strict exactly when a cross-family sentence couples choices that the free product of (4) decouples.*

Proof sketch. Any $p \in \mathcal{D}_{\text{LCN}}$ factorizes as (3) and each conditional $P(\text{ch}_v \mid \text{pa}_v)$ lies in K_v , so $p \in \mathcal{D}_{\text{SE}}$: the inclusion holds. A chain-graph product satisfies every LMC assertion by construction, so a free product in (4) can leave $\mathcal{D}_{\text{LCN}}$ *only* by violating a *sentence*; when every sentence is intra-family, each K_v already encodes every sentence constraining its scope and the free product of compliant families is compliant, giving $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$. A cross-family sentence, by contrast, can be violated by a vertex combination that each family satisfies locally — e.g. on `d4.biting` the free product reaches $P(B \wedge C) \approx 0.9$ against the sentence bound 0.05 — so $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$. $\square$

Consequently any engine that returns a valid bound over $\mathcal{D}_{\text{SE}}$ is automatically a valid *outer* bound over $\mathcal{D}_{\text{LCN}}$: $[\underline{q}_{\text{LCN}}, \bar{q}_{\text{LCN}}] \subseteq [\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$. The gap between the two closes precisely under the Proposition 1 condition.

1.3 The two looseness sources

Following `tighter_approximation.tex`, the gap between a credal-network engine’s output and the LCN truth has two independent origins.

- **Gap A (local-set widening).** Each K_v is computed per family in isolation, so it admits conditionals no global $p \in \mathcal{D}_{\text{LCN}}$ realizes: it ignores sentences on *other* families and the cross-family LMC equalities. Formally $K_v^{\text{true}} \subseteq K_v^{\text{local}}$, often strictly (e.g. the vacuous $[0, 1]$ CPT rows on `alarm`).
- **Gap B (strong-extension widening).** Even with $K_v = K_v^{\text{true}}$, the free product of (4) lets each family pick its vertex independently, violating cross-family sentences. This is exactly the strict inclusion of Proposition 1.

The tightening schemes of `tighter_approximation.tex` target these: D1 (`method="linear-tight"`), D2 (`merge_budget`), D3 (`solver="scip"`) attack Gap A at build time and are free for every credal-network engine; D4 (`coupling="cross-family"`) and D5 (`CredalJT`) attack Gap B at propagation time.

1.4 The bound-character taxonomy

We classify each engine’s returned interval $[\underline{q}, \bar{q}]$ against the truth $[\underline{q}^*, \bar{q}^*]$ (either $\mathcal{D}_{\text{LCN}}$ or $\mathcal{D}_{\text{SE}}$, stated per engine).

Definition 1 (Bound character). *The interval is exact if $[\underline{q}, \bar{q}] = [\underline{q}^*, \bar{q}^*]$; a valid outer bound if $[\underline{q}, \bar{q}] \supseteq [\underline{q}^*, \bar{q}^*]$ (sound: it never excludes a feasible marginal); an inner bound if $[\underline{q}, \bar{q}] \subseteq [\underline{q}^*, \bar{q}^*]$ (not a valid outer bound — it can be too tight); and unreliable if neither containment is guaranteed (a solver or implementation artifact).*

The crucial practical point: an *inner* bound is unsafe as a guarantee — it may claim more certainty than the LCN supports. ApproxLP and Interval BP’s variational mode are inner *by design*; the (now-fixed) ARIEL factor-grouping bug and Interval BP message bugs used to make those engines inner *unsoundly*.

1.5 Three kinds of looseness

It is essential to keep apart *why* an engine’s interval is not the LCN truth:

1. **Definitional** — the engine honestly targets $\mathcal{D}_{\text{SE}}$, and $\mathcal{D}_{\text{SE}} \not\supseteq \mathcal{D}_{\text{LCN}}$ (Gap B) or $K_v^{\text{local}} \not\supseteq K_v^{\text{true}}$ (Gap A). This is sound looseness: the interval is a valid outer bound, just not tight. Closed by D1–D5.
2. **Solver** — the engine targets the right set but uses a local optimizer. This is `ExactInference`’s `solver="local"` (ipopt on a nonconvex bilinear NLP): a local optimum can report a *spuriously inner* (too-tight, unsound) interval. Cured by `solver="global"` (SCIP).
3. **Implementation** — a bug makes the shipped engine depart from its own algorithm. Two were on record and *both are now fixed*: the ARIEL factor-grouping bug (which dropped the largest factor’s sentences, §5.1) and the two Interval BP message bugs (§4.4). We report the *shipped* status, not just the intended theory; with both fixes landed the shipped engines now match their algorithms.

2 Master table

Table 1 is the one-screen summary. “Targets” is the feasible set the engine optimizes; “character” is per Definition 1 against that set; “exact (LCN) when” is the topological condition for the returned interval to equal the $\mathcal{D}_{\text{LCN}}$ truth.

* certified when the SCIP gap $\leq \text{gap_tol}$; on time-out the bound is valid but its optimality is unconfirmed. † Credal CTE is exact over $\mathcal{D}_{\text{SE}}$ on a clean singly-connected sentence-free LCN, but its single shared elimination order falls *strictly inner* the moment a cross-family sentence appears (§4.2). “Shipped” flags the implementation status: *fixed* (ARIEL, §5.1; Interval BP, §4.4 — each had a bug, since corrected), *caveat* (`ExactInference`-local, SCCP).

3 The exact family: targeting $\mathcal{D}_{\text{LCN}}$

3.1 ExactInference: the definitional oracle

`ExactInference` (`exact.py`) solves, for each atom, a min and a max of $P(a)$ (or the ratio $P(a | e)$) over the full 2^n joint subject to *all* sentences and *all* LMC equalities — i.e. directly over $\mathcal{D}_{\text{LCN}}$ of

Table 1: Exactness properties of the LCN marginal-inference engines. In the “exact when” column, “clean s.c.” abbreviates $\mathcal{D}_{\text{SE}}=\mathcal{D}_{\text{LCN}}$, i.e. a singly-connected primal graph with no cross-family sentence.

Engine	Targets	Character	Exact (LCN) when	Cost	Shipped
ExactInference (global)	$\mathcal{D}_{\text{LCN}}$	exact*	always ($\text{gap} \leq \text{tol}$)	2^n ; SCIP	OK
ExactInference (local)	$\mathcal{D}_{\text{LCN}}$	unreliable	— (may over-tighten)	2^n ; ipopt	caveat
CredalJT (D5)	$\mathcal{D}_{\text{LCN}}$	outer; exact if H1–H4	H1–H4 (Thm. 1)	treewidth	OK
Credal VE	$\mathcal{D}_{\text{SE}}$	exact over $\mathcal{D}_{\text{SE}}$	clean s.c.	per-target elim	OK
Credal CTE ($\epsilon=0$)	$\mathcal{D}_{\text{SE}}$	exact over $\mathcal{D}_{\text{SE}}^\dagger$	clean s.c.	two-pass tree	OK [†]
ApproxLP	$\mathcal{D}_{\text{SE}}$	<i>inner</i>	clean s.c., no stall	coord. descent	OK
Interval BP (interval)	$\mathcal{D}_{\text{SE}}$	outer	clean s.c. (tree)	loopy messages	fixed
Interval BP (variational)	$\mathcal{D}_{\text{SE}}$	<i>inner</i>	(inner sample)	mean-field	fixed
Credal IJGP	$\mathcal{D}_{\text{SE}}$	outer-style	clean s.c., $i \geq \text{tw}$	2^i /cluster	OK
ARIEL	$\mathcal{D}_{\text{LCN}}$	outer; exact on s.c.	singly connected	factor messages	fixed
SCCP / SCCP-ARIEL	$\mathcal{D}_{\text{LCN}}$	outer	trivial SCCs	per-SCC NLP	caveat

(2). It is the definitional target, not a member of the credal-network family; in particular it does *not* bound $\mathcal{D}_{\text{SE}}$. Two backends, selected by the `solver` argument of `run`:

- `solver="global"`: SCIP spatial branch-and-bound. Certifies the global optimum, or on time-out returns the best bound with its optimality gap. `self.status` records the per-atom verdict (`confirmed` / `unconfirmed` / `unsolved`).
- `solver="local"` (default): ipopt with a two-phase SLSQP fallback (`find_feasible_points` $\rightarrow$ `optimize_marginal_slsqp`) and multi-restart. Fast, but a *local* method on a nonconvex bilinear-LMC NLP.

Proposition 2 (Global certified; local unreliable). *With `solver="global"` and reported gap $\leq \text{gap_tol}$, **ExactInference** returns the exact $\mathcal{D}_{\text{LCN}}$ bounds. With `solver="local"`, the returned interval is unreliable (Definition 1): it may be spuriously inner (too tight), because a local optimum of the nonconvex NLP can sit strictly inside the true range.*

Proof sketch. The model (2) is exactly $\mathcal{D}_{\text{LCN}}$, so any *globally* optimal min / max equals the true bound; SCIP’s spatial B&B certifies global optimality up to the gap tolerance. ipopt, being a local NLP solver on a nonconvex feasible region (the bilinear LMC equalities are nonconvex), can converge to a stationary point whose objective is strictly interior to the true extremum; the multi-restart + SLSQP fallback mitigates but does not certify. Hence the local backend carries no soundness guarantee. $\square$

Witness (shipped). On `alarm.lcn` the certified bound is $P(A) = [0.087, 0.100]$ (brute-force / SCIP), whereas raw `solver="local"` reports the spuriously tighter $P(A) = [0.092, 0.100]$, $P(E) = [0.050, 0.073]$ — an *inner*, unsound interval. This is the reason the companion notes treat **ExactInference**-local as *not* a reliable oracle and prefer SCIP-backed cross-checks (`verify_scip.py`, `verify_bruteforce.py`); **CredalJT** is often the more trustworthy engine on such instances (§3.2).

3.2 CredalJT (D5): exact at treewidth cost

CredalJT (`cn/junction_nlp.py`) builds one junction (clique) tree from the chain-graph elimination order, gives each cluster C a variable vector q_C over the joint of its atoms, ties adjacent clusters by Lauritzen separator-consistency equalities, and hosts every sentence and LMC equality on a cluster

containing its atoms. It reads each query marginal off a host cluster and optimizes min / max with SCIP. Cost is $\sum_C 2^{|\text{atoms}(C)|}$ — exponential in *treewidth*, not in n .

We compare two feasible sets. The *joint-feasible set* is $\mathcal{F} := \mathcal{D}_{\text{LCN}}$ of (2): full joint distributions p over all 2^n assignments meeting every sentence and LMC constraint. The *cluster-feasible set* $\mathcal{G}$ replaces the one global joint by one local distribution $q_C \in \Delta(2^{|\text{atoms}(C)|})$ per cluster C of the junction tree T , required only to be locally consistent and to satisfy the constraints each cluster hosts:

$$\mathcal{G} = \{(q_C)_{C \in T} : \sum_i q_C[i] = 1 \text{ for each cluster } C \text{ (each } q_C \text{ is a distribution);}$$

$$q_C \upharpoonright S = q_D \upharpoonright S \text{ on every tree edge } (C, D), S = C \cap D \text{ (clusters agree on their separator);}$$

$$\text{every sentence / LMC equality hosted on a cluster holds there}\},$$

where $q_C \upharpoonright S$ is the marginal of the cluster distribution q_C onto the separator atoms S (the “ $M_{C \rightarrow S} q_C$ ” of the implementation — the linear map that sums q_C down to S). An LMC assertion $(X \perp\!\!\!\perp Y \mid S)$ is *hosted* if some cluster contains $X \cup Y \cup S$ (so its equality (1) can be written on that cluster), and *separator-entailed* if S separates X from Y in the chordal graph G^* whose maximal cliques are the clusters.

The read-off functional. Fix a query atom a and evidence e , and let C_a be a cluster containing a together with the evidence atoms (H4 below guarantees one exists). Define the *read-off* ρ_a , the marginal that **CredalJT** reports for a , directly from the host cluster distribution:

$$\rho_a(q) = \sum_{i: a=1 \text{ in state } i} q_{C_a}[i] \quad (\text{no evidence}), \quad \rho_a(q) = \frac{\sum_{i: a=1, e \text{ in state } i} q_{C_a}[i]}{\sum_{i: e \text{ in state } i} q_{C_a}[i]} \quad (\text{with evidence } e).$$

Write $\pi(p) = (p \upharpoonright C)_{C \in T}$ for the *projection* of a joint p onto the clusters. The same formula applied to $\pi(p)$ returns exactly p ’s own marginal, $\rho_a(\pi(p)) = P_p(a)$ (resp. $P_p(a \mid e)$); so the theorem below compares *one* functional ρ_a optimized over $\mathcal{G}$ against the same functional optimized over $\mathcal{F}$.

Theorem 1 (D5 exactness). *Suppose (H1) T has the running-intersection property (RIP; always true by construction); (H2) every LCN sentence is hosted; (H3) every LMC assertion is either hosted or separator-entailed in G^* ; (H4) the query atom and all evidence atoms lie together in some cluster. Then for every such atom a the read-off optimized over the cluster-feasible set equals the true marginal bound optimized over the joint-feasible set,*

$$\min_{q \in \mathcal{G}} \rho_a(q) = \min_{p \in \mathcal{F}} \rho_a(\pi(p)) = \underline{P}(a), \quad \max_{q \in \mathcal{G}} \rho_a(q) = \max_{p \in \mathcal{F}} \rho_a(\pi(p)) = \overline{P}(a),$$

and likewise for a conditional query $P(a \mid e)$. That is, **CredalJT** returns exactly the **ExactInference** bounds.

Proof sketch. Projection $\mathcal{F} \rightarrow \mathcal{G}$ (always valid; needs only H1/H4). Given a joint $p \in \mathcal{F}$, set each cluster distribution to the corresponding marginal, $q_C := p \upharpoonright C$. Then every q_C is a distribution; each separator equality holds because both $q_C \upharpoonright S$ and $q_D \upharpoonright S$ equal $p \upharpoonright S$; and every hosted constraint holds by locality (its value on q_C equals its value on p). Here $q = \pi(p)$, so the read-off identity gives $\rho_a(q) = \rho_a(\pi(p)) = P_p(a)$. Hence $q \in \mathcal{G}$ with the same objective value, so $\min_{\mathcal{G}} \rho_a \leq \min_{\mathcal{F}} \rho_a$ and $\max_{\mathcal{G}} \rho_a \geq \max_{\mathcal{F}} \rho_a$: the cluster interval *contains* the joint interval, i.e. **CredalJT is always a valid outer bound**.

Reconstruction $\mathcal{G} \rightarrow \mathcal{F}$ (needs H2/H3). Conversely, given a separator-consistent family $(q_C) \in \mathcal{G}$, the RIP marginal-consistency theorem (Lauritzen) yields a *unique* joint $p = \prod_C q_C / \prod_{(C,D) \in T} q_S$

(with q_S the common separator marginal) whose cluster marginals are exactly the q_C . We check $p \in \mathcal{F}$. Each *hosted* sentence or LMC equality holds because, by locality, its value on p equals its value on the host cluster’s marginal q_{C_h} , which satisfies it (H2 and the hosted part of H3). Each *separator-entailed* LMC ($X \perp\!\!\!\perp Y \mid S$) holds *automatically*: since p factorizes over T it obeys the global Markov property of the chordal graph G^* , so whenever S separates X from Y in G^* , X and Y are conditionally independent given S *under this reconstructed p* — even though the equality was never imposed on $\mathcal{G}$ (the other part of H3). Thus each cluster family lifts to a joint of the same objective value, giving the reverse inclusion. Both inclusions force equality; by H4 the conditional read-off ρ_a is a functional of the single host cluster C_a , so the same two maps preserve it. $\square$

Three consequences. (a) The proof never mentions chains: exactness is a property of the *junction tree*, not of chains. (b) The projection half is unconditional, so **CredalJT** is *never unsound* — its interval always contains the truth; H2/H3 decide only tightness. (c) On a Markov chain the moral graph is already triangulated and every LMC is single-separator, so H3 holds with treewidth 1. *H3 is sufficient but not necessary*: on **asia** and **polytree** some LMCs are neither hosted nor separator-entailed yet CredalJT is empirically exact (the untailed assertions do not bind); on **smokers** the six large-scope LMCs of the undirected clique $F_1 F_2 F_3$ are unhostable and untailed, H3 fails *and bites*, and CredalJT returns the loose but valid $P(F_1) = [0, 1]$.

4 The credal-network family: targeting $\mathcal{D}_{SE}$

All five engines here consume the same **CredalNetworkVertices**; they differ only in how they propagate the enumerated extreme points, and hence in where they land relative to $[\underline{q}_{SE}, \bar{q}_{SE}]$.

4.1 Credal VE: exact over the strong extension

Credal VE (**cve.py**) computes every atom’s bounds by looping a single-query bucket elimination, each time recomputing an elimination order in which the target is eliminated *last*.

Proposition 3 (CVE exact over $\mathcal{D}_{SE}$). *With `coupling="off"` and exact pruning ($\epsilon=0$), Credal VE returns exactly $[\underline{q}_{SE}(a), \bar{q}_{SE}(a)]$ for every atom, hence a valid outer bound on $\mathcal{D}_{LCN}$.*

Proof sketch. Every potential CVE forms is a finite set of products of local extreme points, i.e. a set of $\mathcal{D}_{SE}$ vertices; combine and marginalize preserve this, and pruning removes only dominated functions, which never attain a componentwise extremum. Because $\mathcal{D}_{SE}$ is a product of finitely many polytopes, each bound is attained at a vertex combination; eliminating the query *last* keeps that combination in the surviving potential through to the read-off, so the reported min / max equals the $\mathcal{D}_{SE}$ extremum. See **cve.exactness.tex** §3. $\square$

Combined with Proposition 1, Credal VE is *exact w.r.t.* $\mathcal{D}_{LCN}$ exactly when $\mathcal{D}_{SE} = \mathcal{D}_{LCN}$ (singly connected, no cross-family sentence). It is the “gold” member of the family: exact over $\mathcal{D}_{SE}$, whereas the others are inner (ApproxLP), outer with loop slack (IBP loopy), or order-sensitive (CTE).

4.2 Credal CTE: exact over $\mathcal{D}_{SE}$, but order-sensitive

Credal CTE (**cccte.py**) is the two-pass (collect + distribute) bucket-tree engine: one shared elimination order produces all marginals at once. With $\epsilon=0$ it is exact over $\mathcal{D}_{SE}$ on a clean singly-connected sentence-free LCN (the shared schedule is a genuine clique tree there). But its single shared order

— deliberately *not* augmented per query, unlike Credal VE — can prune the function carrying an extremal marginal at an intermediate scope where it is dominated. The witness is `d4.biting`: the true $\mathcal{D}_{\text{SE}}$ -and- $\mathcal{D}_{\text{LCN}}$ value is $P(B) = [0.04, 0.72]$ (Credal VE, CredalJT, and two oracles agree), yet Credal CTE reports the strictly *inner* $[0.05, 0.65]$ (and CTE+D4 $[0.05, 0.55]$). So on cross-family instances CTE can fall inside $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$ — *not even a valid outer bound*. See `ccte_exactness.tex`.

4.3 ApproxLP: an inner bound

ApproxLP (`approxlp.py`) reformulates the marginal problem as a multilinear program and solves it by coordinate descent: fix all families but one, pick that family’s best extreme point (a linear subproblem), sweep to convergence.

Proposition 4 (ApproxLP inner). *With `coupling="off"`, ApproxLP returns an interval contained in $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$; in general it is not a valid outer bound on $\mathcal{D}_{\text{LCN}}$.*

Proof sketch. Every state ApproxLP evaluates is a single feasible product joint (each family at one extreme point, or the center), i.e. a point of $\mathcal{D}_{\text{SE}}$, and the objective is evaluated exactly on that precise Bayes net. The reported min/max is therefore a min/max over a *subset* of $\mathcal{D}_{\text{SE}}$ (the coordinate-descent trajectory), hence no smaller / no larger than the global $\mathcal{D}_{\text{SE}}$ extremum: $\underline{q}_{\text{SE}} \leq \underline{q}_{\text{ALP}} \leq \bar{q}_{\text{ALP}} \leq \bar{q}_{\text{SE}}$. See `approxlp_exactness.tex` §2. $\square$

The inner gap is strict only when the nonconvex program makes descent stall at a non-global local optimum. On the instances tested (`d4.biting`, 2-atom chain) it is *sound and tight*: it reproduces the exact $P(B) = [0.04, 0.72]$. Its natural use is the *inner* half of a two-sided bracket: ApproxLP (inner) with Credal VE (exact over $\mathcal{D}_{\text{SE}}$) on the same vertices sandwich the truth and detect stalls.

4.4 Interval BP: outer / inner by mode; exact on trees

Interval BP (`ibp.py`, class `IntervalBP`) is loopy interval message passing over the factor graph. `method="interval"` passes per-state intervals and is an *outer* approximation of $[\underline{q}_{\text{SE}}, \bar{q}_{\text{SE}}]$; `method="variational"` runs mean-field over a finite sample of vertex combinations and is an *inner* approximation.

Proposition 5 (IBP interval outer; exact on tree). *With `method="interval"`, Interval BP returns a valid outer bound over $\mathcal{D}_{\text{SE}}$. When the factor graph is a tree (Markov chain, tree, polytree) it is exact over $\mathcal{D}_{\text{SE}}$; combined with Proposition 1 it is then exact over $\mathcal{D}_{\text{LCN}}$.*

Proof sketch. Each factor→child update ranges over all local extreme points and all corner *distributions* of the incoming messages, so every $\mathcal{D}_{\text{SE}}$ marginal is contained in the message interval; a conditional factor sends the vacuous $[0, 1]$ upward, excluding no parent marginal; variable→factor messages only intersect. On a tree there is a unique path between factors, so no message feeds back and no correlated pair’s corners over-cover, giving the exact $\mathcal{D}_{\text{SE}}$ range. See `ibp_exactness.tex`. $\square$

Shipped status: fixed. Two message bugs previously made the interval mode inner and unsound — a spurious normalized-likelihood upward message (collapsing an unconstrained root to $[0.5, 0.5]$) and an invalid component-wise corner enumeration. Both are now corrected; the shipped engine reproduces Credal VE on the 2-atom chain ($P(x_0) = [0.30, 0.50]$, was $[0.5, 0.5]$) and on `d4.biting` ($P(B) = [0.04, 0.72]$, was $[0.05, 0.65]$). One residual limitation is architectural: the intersection message algebra performs no Bayesian *upward* update, so evidence on a child does not shift a parent’s posterior — for conditional queries with evidence on descendants use Credal VE / CredalJT.

4.5 Credal IJGP: exact at $i\text{-bound} \geq \text{treewidth}$

Credal IJGP (`ijgp.py`) runs iterative join-graph propagation with a mini-bucket `i_bound` capping cluster size, forward+backward passes to a message-change threshold.

Proposition 6 (IJGP exact at large i -bound). *When $i\text{-bound} \geq$ the treewidth of the credal network, the join graph is a genuine junction tree and Credal IJGP is exact over $\mathcal{D}_{\text{SE}}$; with $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ it is exact over $\mathcal{D}_{\text{LCN}}$. For smaller $i\text{-bound}$ it is an outer-style approximation, tunable by the i -bound.*

Proof sketch. At $i\text{-bound} \geq \text{treewidth}$ no mini-bucket partitioning is forced, so the join graph carries no cycle and message passing is exact elimination over the enumerated vertices (the same argument as Interval BP on a tree, one level up at cluster granularity). Below treewidth, mini-bucket splitting introduces the loopy slack that the i -bound trades against cost $2^i/\text{cluster}$. $\square$

5 Structural and hybrid engines: targeting $\mathcal{D}_{\text{LCN}}$ by decomposition

5.1 ARIEL: structural LMC preservation

ARIEL (`ariel.py`) runs interval message passing on the bipartite factor graph whose factor nodes group sentences by shared atom scope.

Proposition 7 (ARIEL exact on singly-connected). *On a singly-connected LCN, ARIEL returns the exact $\mathcal{D}_{\text{LCN}}$ marginal bounds. For every LMC assertion $(X \perp\!\!\!\perp Y \mid S)$: if $X \cup Y \cup S$ fits in one factor, the bilinear equality is imposed in that factor’s local NLP; otherwise the assertion factorizes into single-separator independencies along the unique factor path, each realized exactly by an interval message across that separator. The interval summary is lossless precisely because a tree has no second path along which a discarded joint correlation could matter.*

Proof sketch. Structural LMC preservation on a tree factor graph (`ariel_exactness.tex` Prop. 1). On the shipped chain/tree/polytree, *no* LMC fits inside a single factor, so none is imposed as an explicit constraint, yet the assertions survive structurally along the tree paths, giving exactness. On a loopy factor graph the interval message across a boundary variable loses the joint correlation that a second path needs, so ARIEL is a valid outer bound but not tight (the coupled-diamond example, $P(D) = [0.316, 0.684]$ vs exact $[0.316, 0.594]$). $\square$

Shipped status: fixed. An earlier factor-graph construction (`factor_graph.py`, `FactorGraph.make_factor_nodes`) had an off-by-one labelling bug: it created the first factor before the grouping loop and then reused the label `f1` for the next new-scope factor, silently overwriting — and dropping the sentences of — the first, largest-scope factor (sentences are sorted by decreasing scope). The shipped ARIEL therefore returned vacuous $[0, 1]$ interior marginals on chain/tree/polytree, contradicting Proposition 7. The bug is now fixed (each new-scope factor gets a fresh post-incremented label). Verified: on the two-atom chain the fixed engine returns $P(x_0) = [0.30, 0.50]$, $P(x_1) = [0.40, 0.68]$, matching Credal VE / `ExactInference`; all tests pass. See `ariel_exactness.tex`.

5.2 SCCP / SCCP-ARIEL: exactness on the acyclic condensation

The SCC-factor-graph engines (`sccp.py`, `sccp_ariel.py`) contract each strongly connected component of the structure graph into a super-node, giving an acyclic condensation DAG, and run two-pass (collect + distribute) belief propagation on it. On the condensation the propagation is exact elimination; the only inexactness is *inside* a cyclic super-node, whose local nonconvex NLP

is solved by `ipopt` (`sccp.py`, `local` $\Rightarrow$ the same reliability caveat as `ExactInference-local`) or approximately by ARIEL loopy BP (`sccp_ariel.py`). Hence: when every SCC is trivial (a singleton, whose local problem is an LP), the two-pass propagation is exact over $\mathcal{D}_{\text{LCN}}$; a non-trivial SCC introduces solver looseness (v1) or approximation (v2). See `scc_factor_graph.tex`.

6 Topology cross-cut

Table 2 reads the master table along the other axis: for each canonical topology, the character of each engine’s shipped output. “ex” = exact, “out” = valid outer bound (loose), “in” = inner (unsound as a guarantee), “unrel” = solver-unreliable.

Table 2: Bound character by topology (shipped engines; unconditional marginals).

Topology	Exact-glob	CredalJT	CVE	CTE	ApproxLP	IBP-int	IJGP [‡]
Markov chain (chain)	ex	ex	ex	ex	ex	ex	ex
Tree (tree)	ex	ex	ex	ex	ex	ex	ex
Polytree (polytree)	ex	ex [†]	ex	ex	ex	ex	ex
Chain graph, compound (alarm)	ex*	ex	ex	ex	ex	ex	ex
Cross-family sentence (d4_biting)	ex	ex	ex	in	ex	ex	out
Loopy DAG	ex	out	out [§]	out [§]	in [§]	out	out
Undirected clique (smokers)	ex	out	out [§]	out [§]	in [§]	out	out

* certified-global is exact; `ExactInference-local` over-tightens here ($P(A) = [0.092, 0.100]$ vs $[0.087, 0.100]$) — *unrel*. [†] H3 fails but CredalJT is empirically exact (untailed LMCs do not bind). [‡] IJGP shown at a small i -bound; at i -bound $\geq$ treewidth it matches CredalJT. [§] On a loopy/undirected-clique LCN $\mathcal{D}_{\text{SE}} \supsetneq \mathcal{D}_{\text{LCN}}$; the $\mathcal{D}_{\text{SE}}$ -family entries are exact-over- $\mathcal{D}_{\text{SE}}$ but only *outer* (CVE/CTE/IBP/IJGP) or *inner* (ApproxLP) over $\mathcal{D}_{\text{LCN}}$. On **smokers** even CredalJT is outer-only ($P(F_1) = [0, 1]$). For cross-family / loopy instances the reliable exact route is `ExactInference-global` or `CredalJT` where H1–H4 hold.

7 Reliability and verification

Three verifiers cross-check the engines against ground truth (`lcn/inference/marginal/`):

- `verify_scip.py` — certified global (SCIP spatial B&B) marginal bounds; the reference oracle when it terminates within the gap tolerance.
- `verify_bruteforce.py` — LMC-aware brute-force enumeration cross-check; found the `ExactInference-local` over-tightening on **alarm**.
- `verify_lmc_factorization.py` — solver-free check that the credal network’s product joints satisfy the LMC (certifies $\mathcal{D}_{\text{SE}} = \mathcal{D}_{\text{LCN}}$ on chain/tree/polytree, i.e. Gap B = 0).

The reliability lessons carried into Table 1: `ExactInference-local` is a *local* solver (use `solver="global"` when a guarantee is needed); the ARIEL factor-grouping bug and the two Interval BP message bugs are all *fixed*, so those shipped engines now match their algorithms; Credal CTE is order-sensitive and can be inner on cross-family instances. When two engines that both claim “exact over $\mathcal{D}_{\text{SE}}$ ” (CVE and CTE) disagree, the disagreement localizes an order-sensitivity or a bug, not a modeling difference.

8 Decision guide

- **Need a certified guarantee, small n :** `ExactInference` with `solver="global"`. Never trust `solver="local"` as a guarantee.
- **Need exactness at scale, bounded treewidth:** `CredalJT` (D5) — exact under H1–H4 at cost $\sum_C 2^{|C|}$, always a valid outer bound otherwise. This is the recommended exact engine for chain-graph LCNs, and is often more trustworthy than `ExactInference-local` (`alarm`).
- **All singleton marginals over $\mathcal{D}_{SE}$, exactly:** `Credal VE`. Prefer over `Credal CTE` when a cross-family sentence is present (CTE can fall inner).
- **A two-sided sandwich of the truth:** pair `Credal VE` (outer over $\mathcal{D}_{SE}$) with `ApproxLP` (inner over $\mathcal{D}_{SE}$) on the same vertices; equal endpoints certify the $\mathcal{D}_{SE}$ bound.
- **Fast, singly-connected:** `Interval BP` `method="interval"` or `ARIEL` — both exact on trees now that their respective bugs are fixed.
- **Tighten any $\mathcal{D}_{SE}$ -family engine toward $\mathcal{D}_{LCN}$:** apply the build-stage schemes D1 (`method="linear-tight"`), D2 (`merge_budget`), D3 (`solver="scip"`) first — free for all of them — then D4/D5 for cross-family coupling.

The six companion notes hold the per-engine detail: `cve_exactness.tex`, `ccte_exactness.tex`, `ibp_exactness.tex`, `approxlp_exactness.tex`, `ariel_exactness.tex`, and `tighter_approximation.tex` (the D1–D5 taxonomy and the full D5 theorem).